

Contenido

Capítulo 1. Arquitectura y organización del proyecto.	1
Frameworks, herramientas y lenguajes de programación.	1
Backend.	1
Frontend.	1
Requerimientos	1
Arquitectura y patrón de diseño.	1
Comunicación entre el controlador y el modelo	2
Inyección de dependencias	7
Organización de carpetas y archivos	8
Carpetas	8
Archivos	10
Capítulo 2. Diseño.	10
Layouts	10
Componentes	11
Capítulo 3. Autenticación.	11
Usuarios.	11
Adición de campos a la tabla de usuarios	12
Registro de usuarios	12
Métodos de autenticación	12
Autenticación mediante cookies	12
Autenticación mediante tokens usando OAuth 2.0	12
Capítulo 4. Autorización.	14
Creación de opciones de acceso	14
Creación de categorías	15
Capítulo 5. Manejo de la base de datos.	15
Migraciones	15
Creación de tablas.	15
Modificación de una tablas.	16
Capítulo 6. Manejo de respuestas HTTP y errores.	17
Gestión de respuestas HTTP	17
Gestión de errores	18

Capítulo 7. Manejo de mensajes y notificaciones.	18
Mensajes desde el backend	19
Mensajes desde el frontend	19
Capítulo 8. Convenciones del código.	20
Capítulo 9. Manejo del Frontend	20
Gestión de paquetes	20
Empaquetado (Module Bundler)	20
Configuración de webpack para empaquetar módulos	21
Capítulo 10. Manejo de Reportes	22
Opciones y reportes	22
Visor de reportes	23
Capítulo 11. Manejo de formularios	23
La vista para el formulario	23
El módulo index.js	24
Capítulo 12. Manejo de grid de datos	25
Selección de filas	25
Referencias	26

Capítulo 1. Arquitectura y organización del proyecto.

Frameworks, herramientas y lenguajes de programación.

Backend.

- ASP.NET MVC 5
- C#
- Entity Framework
- SQL Server
- Microsoft Identity
- Visual Studio 2019

Frontend.

- HTML
- JavaScript
- SASS
- Bootstrap 5
- NPM
- Webpack

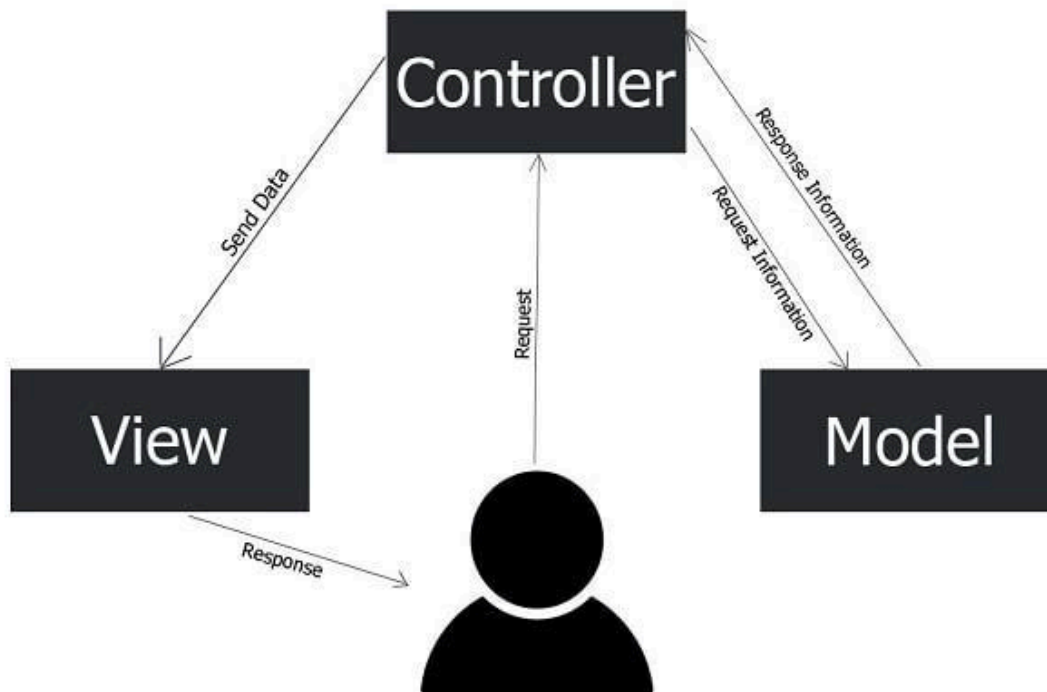
Requerimientos

Para hacer uso de la plantilla, se debe instalar Visual Studio 2019 (en cualquiera de sus versiones) y Node.js (Para poder trabajar con el administrador de paquetes NPM)

Arquitectura y patrón de diseño.

El proyecto utiliza el patrón MVC para separar las capas de proyecto, modelo de datos, lógica de negocios y presentación.

Model-View-Controller



Comunicación entre el controlador y el modelo

Para favorecer una arquitectura limpia y reducir el acoplamiento entre las partes de la aplicación, **es importante evitar la colocación de lógica de acceso a los datos y reglas de negocio directamente en los métodos del controlador**. Por el contrario, se recomienda colocar capas intermedias entre el **controlador** y el **modelo**. Estas capas deben tener responsabilidades específicas: El **acceso a los datos** y la **aplicación de reglas de negocio**.

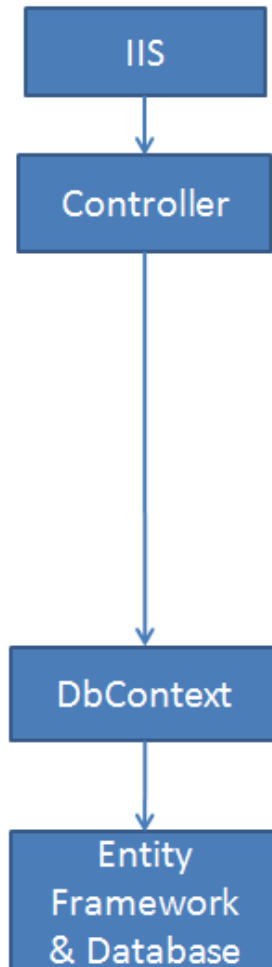
Patrón de repositorio

Hay varios enfoques para abordar esto, siendo una de las más conocidas, el Unit Of Work Repository Pattern¹, abreviado como patrón de repositorio. En el enlace anterior, Microsoft brinda un ejemplo de cómo implementarlo en un proyecto MVC. La siguiente imagen resume la comparación entre usar o no el patrón de repositorio.

¹ [Unit of work pattern repository](#)

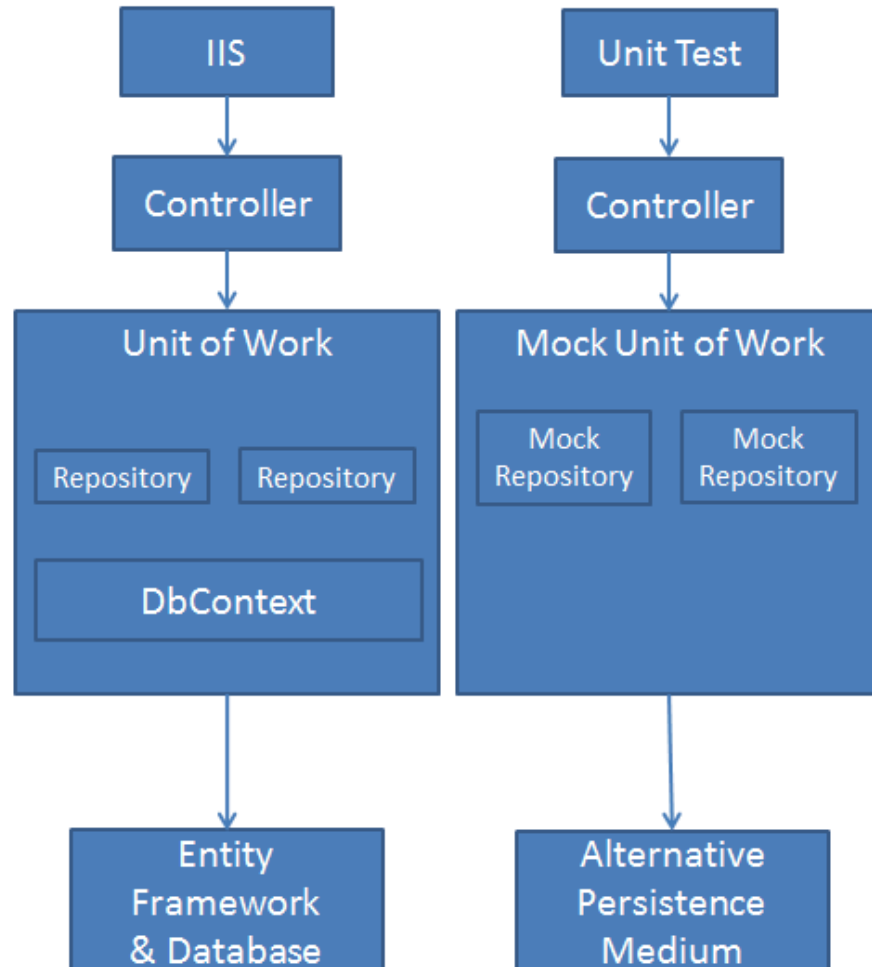
No Repository

Direct access to database context from controller.



With Repository

Abstraction layer between controller and database context. Unit tests can use a custom persistence layer to facilitate testing.



El ejemplo anterior implementa un repositorio para cada controlador, aunque existen otro enfoque donde se implementa un **repositorio genérico** para consultar o modificar cualquier tipo de entidad. Adicionalmente, este modelo **se suele adicionar otra capa, la capa de servicio** que se ubica entre el repositorio y el controlador y se usa exclusivamente para aplicar reglas de negocios a los datos que se pasan o se consultan de la capa de repositorio.

Inconvenientes del patrón de repositorio con Entity Framework

El enfoque de repositorio y servicio, muy correcto en la teoría, presenta 2 inconvenientes principales:

- **Consultas complejas:** Un repositorio genérico funciona bien con consultas simples, como insertar una entidad, actualizar una entidad, obtener una entidad, etc. Operaciones muy simples que se hacen sobre una tabla en base de datos.

Sin embargo, en las aplicaciones reales, lo normal es hacer operaciones más complejas, consultas que se basan en la unión de múltiples tablas con variedad de condiciones dinámicas, ordenamiento, agrupamiento, actualizaciones condicionales masivas, inserción masiva de datos, etc. Estas consultas tan personalizadas, hacen ineficiente el patrón de repositorio genérico. Este problema se puede resolver con otros enfoques como el patrón CQRS.

- **Código redundante:** asumiendo que trabajamos con Entity Framework (abreviaremos como EF), si analizamos, nos damos cuenta de que el mismo, ya aplica un patrón de repositorio genérico. Si nosotros aplicamos un patrón de repositorio encima de EF, la pregunta evidente sería ¿Es necesario usar el patrón de repositorio con EF? Si lo usamos, ¿estaríamos escribiendo una capa de abstracción redundante? Y si añadimos **la capa de servicio**, ¿no estaríamos extendiendo aún más el código redundante?

Las respuestas a estas preguntas son tema de un debate con argumentos a favor y en contra que exceden el alcance de esta guía, sin embargo, vamos a ver un ejemplo: Supongamos que queremos obtener un **estudiante**, de una entidad **Students**, con EF, es algo que podemos hacer en 1 línea:

```
return context.Students.Find(id);
```

Si aplicamos un patrón con capa de repositorio, más capa de servicio, tendríamos que hacer lo siguiente:

1. Una clase **StudentRepository**, con un método para obtener el estudiante.

```
public class StudentRepository : IStudentRepository
{
    public Student GetStudentByID(int id)
    {
        return context.Students.Find(id);
    }
}
```

2. La interfaz correspondiente **IStudentRepository** que debe implementar el repositorio anterior.
3. Una clase de **StudentService** que debe tener un método para obtener el estudiante. Este método es llamado desde el controlador. Luego el método del servicio llama al método del repositorio, que finalmente es el que llama al método **genérico Find** de **Entity Framework**, que obtiene el estudiante del **modelo** para luego iniciar toda la cadena de devolución hacia atrás, hasta llegar al controlador.

```
public class StudentService : IStudentService
{
    private readonly IStudentRepository _studentRep;
    public StudentService(IStudentRepository studentRep)
```

```

{
    _studentRep = studentRep;
}
public Student GetStudentByID(int id)
{
    return studentRep.GetStudentByID(id);
}
}

```

4. La interfaz `IStudentService` que debe implementar el servicio anterior.

¿Qué inconveniente tiene hacer todo esto?

- Estamos **haciendo varias capas de abstracción** (varias clases, interfaces, métodos) **para** hacer una operación que en principio **era sencilla y se resolvía en una línea**.
- Si no se aplica ninguna regla de negocio a la entidad estudiante devuelta, **la capa de servicio se vuelve redundante**.
- Agregar más capas a una aplicación **añade más complejidad al mantenimiento** de esta y **no siempre resulta ventajoso**.
- La idea de que un repositorio debe tener unos **métodos genéricos** de acceso a los datos, y que los métodos que hacen operaciones más específicas deben depender de los métodos genéricos (que por ejemplo devuelven **IQueryable**) puede añadir **problemas de rendimiento** que no sucederían si nos enfocamos en hacer un método con una consulta específica y optima.

Implementación propuesta para el patrón de repositorio

Para abordar los inconvenientes del modelo anteriormente propuesto, en esta plantilla se aplicaron las siguientes ideas:

- Olvidarse del repositorio genérico o forzarnos a tener métodos genéricos. Mejor escribir métodos específicos y óptimos para la necesidad de cada consulta.
- Fusionar la capa de repositorio y servicio en una sola capa, pero separando ambos conceptos en clases diferentes. Por ejemplo, ahora solo tendríamos la Capa **StudentRepository** (con su respectiva interfaz).

```

public class StudentRepository : IStudentRepository
{
    public Student GetStudentByID(int id)
    {
        return context.Students.Find(id);
    }
}

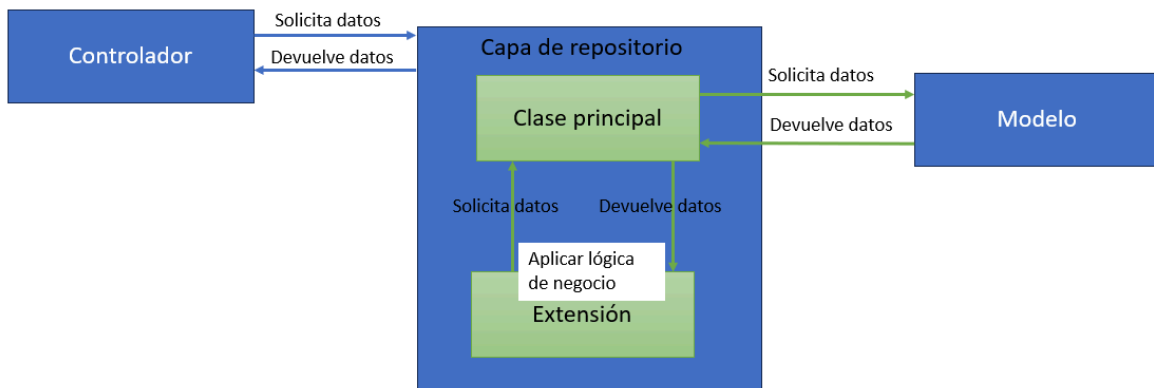
```

¿Y qué sucede si tenemos que aplicar lógica de negocio a los datos que nos devuelve el modelo?

La solución es crear una clase de extensión **StudentExtension** donde se aplica la lógica de negocios:

```
static class StudentExtension
{
    public CustomStudent GetCustomStudent (this
    IRepository studentRep, int id)
    {
        var student = studentRep.GetStudentByID(id);
        var customStudent = // Se aplican reglas de negocio
        // sobre "student"
        return customStudent;
    }
}
```

Y luego sería el método **CustomStudent** el que se llamaría desde el controlador.



¿Ventajas? Sí...

- Tenemos métodos específicos para las consultas, donde nos podemos enfocar en el rendimiento.
- Reducimos capas de la aplicación evitando complejidad para el mantenimiento y la escalabilidad.
- A pesar de eliminar la capa de servicio, todavía podemos separar nuestra lógica de negocios en un lugar separado que pertenece a la misma capa.
- Si **no** necesitamos aplicar lógica de negocios, no creamos clases de extensión. El controlador **llama al método del repositorio y obtiene los datos**. Si necesitamos aplicar lógica de negocios, creamos las clases de extensión para mantener esa lógica separada. El controlador **llama el método de la extensión y obtiene los datos**. Para el controlador **es irrelevante si el método es de la clase original o de extensión**, el solo invoca a una capa (la de repositorio) y obtiene datos.

¿Desventajas? También...

Desafortunadamente no todo es positivo con este enfoque. El hecho de crear repositorios con métodos específicos significa que es muy probable que tengamos repositorios con muchos métodos, que crecen más y más, lo que se conoce como el anti-patrón God Object². Esto se puede

² [Anti Patrón God Object](#)

solucionar con algún otro concepto de ingeniería de software, pero que a su vez añadiría más complejidad a la aplicación, que era lo que se quería evitar inicialmente, pero que a la vez se hace para mantener las buenas prácticas en la arquitectura de la aplicación. **El balance final entre complejidad y buenas prácticas dependerá de las necesidades de cada aplicación.**

Inyección de dependencias

Con el objetivo de reducir el acoplamiento entre los módulos, en la medida que sea posible, se aplica el patrón de diseño de Inyección de dependencias³. Para realizar esto, se hace uso de la librería Unity Container para MVC.

Veamos un caso. Si tengo una clase, por ejemplo, un controlador llamado **ReportController**, que necesita hacer uso de un servicio de reportes, creado en una clase **CrystalReportService**, que retorna un archivo PDF con el reporte. Normalmente haríamos esto en el controlador:

```
public class ReportController : Controller
{
    public ActionResult GetStudentsReport()
    {
        var crs = new CrystalReportService();
        var reportFile = crs.GetReportFileById(1);
        return File(reportFile);
    }
}
```

Esto genera un acoplamiento que trae consigo los siguientes problemas:

- A inicializar el objeto (la línea resaltada en amarillo), el método del **ReportController**, es dependiente de que primero se ejecute código específico de **CrystalReportService**. Recordemos que **CrystalReportService** puede tener un constructor que ejecute lógica propia de Crystal Reports. Al final el **ReportController**, solo necesita un método que le devuelva el archivo y su funcionamiento no debería depender de código específico de otra clase.
- La dependencia explicada en el punto anterior dificulta las pruebas unitarias. No puedo probar el controlador de forma separada, porque su ejecución está acoplada a otra clase.
- Si tengo que cambiar, el servicio para usar otro proveedor, por ejemplo, un **PowerBiReportService**, tengo que **cambiar la inicialización del objeto en cada línea donde se use el servicio de reportes**, porque ya sería una clase diferente.

Existe una solución...

El quinto principio SOLID⁴, de inversión de dependencias, no dice lo siguiente:

Las abstracciones no deberían depender de los detalles. Los detalles (implementaciones concretas) deben depender de abstracciones. (Wikipedia, s.f.)

³ [Patrón de inyección de dependencias](#)

⁴ [Principios SOLID](#)

Siguiendo con el ejemplo anterior, lo primero que tenemos que hacer es una interfaz general para el servicio de reportes (una abstracción). Podemos verlo como un “**contrato**”, que todas las clases de servicio de reporte (implementación concreta) deben cumplir. En otras palabras, cada clase para reportes (que use Crystal, PowerBi, etc.), **debe tener los mismos métodos y parámetros** que tiene la interfaz. Quedaría así:

```
public interface IReportService
{
    byte[] GetReportFileById(int reportId);
}
```

Luego, nuestra clase **CrystalReportService**, debe implementar la interfaz anterior, de la siguiente manera:

```
public class CrystalReportService : IReportService
{
    public byte[] GetReportFileById(int reportId)
    {
        // Lógica usando Crystal Report para obtener el PDF
        // return pdfFile;
    }
}
```

Y finalmente vamos a usarlo en el controlador, pero recordemos que no vamos a crear objetos y **no vamos a depender de implementación concreta**, si no de la **interfaz** (la abstracción). Vamos a “inyectar” la dependencia en el controlador, aquí entra en juego Unity Container que va a gestionar esto por nosotros. Unity va a actuar como un contenedor donde registramos nuestras interfaces y el tipo correspondiente que vamos a usar, de la siguiente manera:

Para esta plantilla, en el archivo App_Start/UnityConfig.cs colocamos lo siguiente:

```
container.RegisterType<IReportService,
CrystalReportService>(TypeLifetime.Transient);
```

Finalmente, la dependencia se inyecta en el constructor de nuestro controlador y se usa para llamar al método y obtener el reporte.

```
public class ReportController : Controller
{
    private readonly IReportService _repService;
    public ReportController(IReportService repService)
    {
        _repService = repService;
    }
    public ActionResult GetStudentsReport()
    {
        var reportFile = _repService.GetReportFileById(1);
        return File(reportFile);
    }
}
```

```
}  
}
```

De esta manera desacoplamos nuestros componentes y resolvemos todos los problemas anteriores. Si necesitáramos usar otro proveedor de reportes, por ejemplo, **PowerBIReportService**, solo tenemos que hacer el cambio en el archivo de configuración de Unity Container.

Organización de carpetas y archivos

Carpetas

A continuación, se describe la función de las principales carpetas y subcarpetas en las que se organiza el proyecto:

- **App_Start:** Contiene las clases que se encargan de configurar parámetros y ejecutar cierta lógica necesaria en el inicio de la aplicación.
- **Controllers:** Clases que sirven de intermediario entre los elementos de la interfaz de usuario y los modelos. Reciben y responden las peticiones HTTP.
- **dist:** Contiene el código de JavaScript y CSS ya compilado desde la carpeta src, mediante el empaquetador de módulos webpack.
- **Extensions:** Contiene clases que extienden la funcionalidad de otras clases o librerías, sin tener que modificar las clases originales.
- **Filters:** Contiene los filtros⁵, una parte fundamental de MVC 5 que permite escribir lógica personalizada que se ejecuta antes o después de la ejecución de acciones de cada controlador. Aquí se encuentran los filtros personalizados definidos por el programador.
- **Helpers:** Contiene clases que sirven para la generación de HTML. Los métodos de los helpers se invocan desde las vistas y permiten renderizar componentes HTML.
- **Migrations:** Se almacenan las migraciones. Las migraciones son el equivalente a las versiones de control de código fuente, pero aplicados a la base de datos. Cada clase contiene las instrucciones necesarias que se traducirán el SQL para realizar cambios en la estructura de la base de datos (crear o modificar tablas, procedimientos, etc.).
Adicionalmente, contiene el archivo *Configuration.cs*, donde se puede insertar datos iniciales en las tablas.
- **Models:** Contiene las clases que se utilizan como modelos. Se dividen en 2 categorías:
 1. **Entity Models:** Se ubican en la subcarpeta “Tables”. Contiene las clases que sirven como modelo para las entidades de base de datos, en este caso, las tablas.
 2. **Data Transfer Object:** Abreviado como DTO⁶, son clases con atributos personalizados que sirven como modelo base de transferencia de datos entre las capas de la aplicación. Por ejemplo, transferir datos del controlador a la vista y viceversa, transferir datos del repositorio hacia un reporte, etc.
- **Providers:** Contiene la lógica para proveedores de autenticación. Inicialmente se incluye el archivo “ApplicationOAuthProvider” que se encarga de proveer la autenticación mediante tokens “Bearer” usando el protocolo de autenticación OAuth 2.0⁷.

⁵ [Filtros en ASP.NET MVC](#)

⁶ [Patrón de diseño DTO](#)

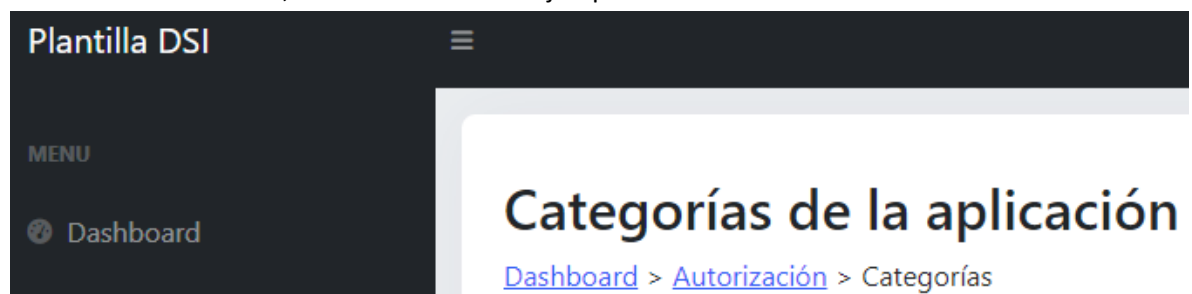
⁷ [Protocolo OAuth 2.0](#)

- **Repositories:** Contiene las clases de repositorio, que consisten en una abstracción de la lógica de acceso a los datos. De esta manera, otras capas de la aplicación como los controladores pueden consultar o modificar los datos. Existen 3 subcarpetas:
 1. **Interfaces:** Interfaces a partir de las cuales se implementan los repositorios. **Todo repositorio debe estar basado en una interfaz, y será dicha interfaz (no la clase de repositorio en sí) la que utilizarán las otras capas de la aplicación para acceder a los datos**, respetando así el principio de inversión de dependencia⁸.
 2. **Entity Framework:** Repositorio que utiliza Entity Framework para el acceso a datos. **Solo deben tener lógica para obtener o modificar los datos de la fuente**, en este caso, la fuente debe ser una base de datos SQL.
 3. **Extension:** Contiene clases con métodos de extensión para manipular o transformar los datos mediante reglas de negocio. **Únicamente se debe trabajar con datos ya obtenidos de la fuente (base de datos en este caso)**, mediante los métodos del repositorio principal.
- **Services:** Clases de que actúan como servicios dentro de la arquitectura de la aplicación.
- **src:** Contiene código relacionado al frontend, principalmente archivos de JavaScript y SASS. Aquí se coloca el código “fuente” que posteriormente será empaquetados en módulos para ser utilizado en la aplicación.
- **Views:** Las vistas donde se genera el HTML para la interfaz de usuario. Las mismas están divididas por carpetas, por cada controlador le corresponde una subcarpeta de vistas. Además, esta la carpeta “Shared” que tiene vistas que se comparten con propósitos de reutilización.

Archivos

Se describen la funcionalidad de algunos archivos importantes del proyecto.

- **Mvc.sitemap:** Se usa para definir una estructura jerárquica de las diferentes pantallas o vistas de la aplicación. Esta estructura se utiliza para generar el elemento de la interfaz de usuario conocido como, “breadcrumb”. Por ejemplo:



- **package.json:** Archivo donde se definen todos los paquetes para el frontend instalados mediante NPM.
- **web.config.json:** Archivo que se usa para definir parámetros y constantes que se pueden usar en cualquier parte de la aplicación. Es accesible desde el frontend y el backend. **Nunca coloque información confidencial o de seguridad en este archivo, porque se puede leer desde el cliente. Para guardar este tipo de información, use el archivo Web.Config.**

⁸ [Principio de inversión de dependencia](#)

- **WebConfig.cs:** Una clase de C# para definir el tipado que tendrá el archivo web.config.json. Además, contiene los métodos para leer los valores de dicho archivo.

Capítulo 2. Diseño.

Lo elementos relacionados al tópico de diseño se encuentran en la carpeta “Views”. Se utiliza el motor de vistas predeterminado (Razor) para crear vistas maestras (Layouts). Estas a su vez están formadas por componentes, que se crean utilizando vistas parciales, para que puedan ser reutilizables.

Layouts

Son vistas maestras, plantillas usadas como base de diseño para crear vistas específicas de las pantallas del aplicativo. Se incluyen las siguientes:

- **_Authentication.cshtml:** Se usa como base para las páginas relacionadas a la autenticación, como registro, inicio de sesión, etc.
- **_Dashboard.cshtml:** Se usa como base para crear las pantallas de la aplicación dentro del espacio de trabajo de un usuario autenticado.
- **_DashboardForm.cshtml:** Igual que la anterior, solo que incluye algunos scripts para la validación de formularios para no tener la necesidad de importarlos desde un archivo de JavaScript.
- **_Error.cshtml:** Se usa como base para las páginas genéricas de errores.
- **_Landing.cshtml:** Se usa como base para la página de inicio de la aplicación.

Es posible crear más Layouts según las necesidades de la aplicación.

Importante: Para garantizar el correcto funcionamiento de la aplicación, en los nuevos Layouts, debe incluir los siguientes 3 componentes dentro del encabezado:

```
<html lang="es">
  <head>
    @Html.Partial("~/Views/Shared/Components/Head/_Meta.cshtml")
    @Html.Partial("~/Views/Shared/Components/Head/_Title.cshtml")
    @Html.Partial("~/Views/Shared/Components/Head/_ThemeScripts.cshtml")
  </head>
```

Componentes

Se utilizan vistas parciales para crear componentes reutilizables en las diferentes vistas. Esto nos permite también modularizar el diseño de nuestras vistas. En la carpeta Components, podemos encontrar componentes para el footer, barra de navegación superior, barra de menú lateral, entre otros.

Capítulo 3. Autenticación.

Para el proceso de autenticación se utiliza la librería de Microsoft Identity, que provee las herramientas necesarias para autenticar mediante cuentas de usuario individuales, proveedores externos y mediante Tokens.

Usuarios.

En la base de datos se puede encontrar la tabla **AspNetUsers**, donde se almacenan la información de los usuarios registrados. Esta tabla trae campos predeterminados. **No elimine ni cambie el nombre ningún campo predeterminado, porque son requeridos para el funcionamiento de Microsoft Identity.** No existe ningún inconveniente en agregar nuevos campos a la tabla. Para esta plantilla se han agregados los siguientes campos:

- **CardId:** Reservado para un número de indentificación del usuario, por ejemplo la cédula.
- **IsActive:** Activar o inactivar un usuario. Un usuario inactivo no puede acceder a ningún recurso que requiera autenticación.
- **CanRequestApiToken:** booleano para indicar si se permite que un usuario solicite tokens para acceso mediante API.
- **ExpirationApiToken:** El tiempo de duración (en días) que se les asignará a los tokens para API, en el momento de su emisión.

Nota: Tampoco debe eliminar estos campos.

Adición de campos a la tabla de usuarios

No se recomienda agregar campos a la tabla directamente mediante SQL. Lo recomendable es agregar los campos al **modelo de Usuarios** y luego crear y aplicar una **migración**. El modelo de usuarios se encuentra en Models/IdentityModels.cs, en la clase **ApplicationUser**.

```
public class ApplicationUser : IdentityUser<int, CustomUserLogin, CustomUserRole, CustomUserClaim>
{
    [StringLength(20)]
    public string CardId { get; set; }
    public bool IsActive { get; set; }
    public bool CanRequestToken { get; set; }

    // Agregar nuevo campo aquí
}
```

Registro de usuarios

Microsoft Identity incluye vistas y controladores para procesos de autenticación. Estos componentes se han adaptado al diseño de esta plantilla. En el caso del proceso de registro, se puede realizar en el enlace <https://tudominio/Account/Register>

Métodos de autenticación

Esta plantilla cuenta con 2 métodos de autenticación, pero se pueden agregar más.

Autenticación mediante cookies

La autenticación tradicional a través del navegador mediante una pantalla de inicio de sesión con usuario y contraseña.

Autenticación mediante tokens usando OAuth 2.0

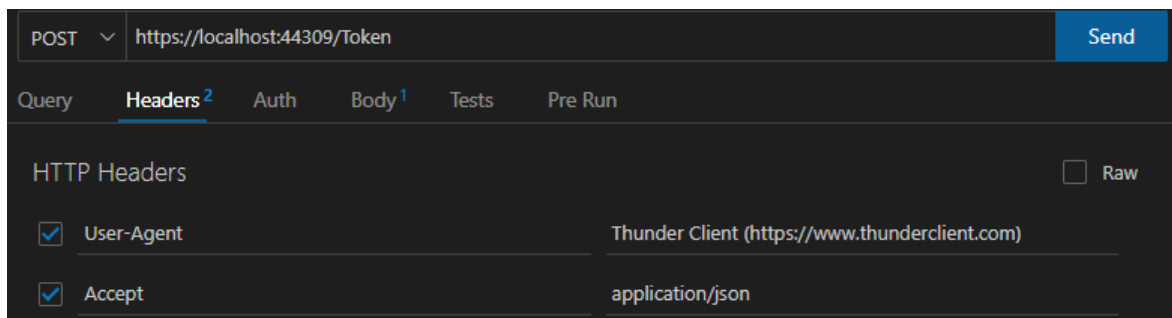
Se puede autenticar un usuario mediante tokens, y acceder a los recursos de este, con lo cual el aplicativo web se puede convertir en una especie API REST básica. El primer paso sería solicitar un token.

Solicitud de token Bearer

Para solicitar un token, se debe realizar una solicitud HTTP a la dirección <https://tudominio/Token>. A continuación se muestra la configuración requerida para realizar la solicitud:

Importante: Antes de solicitar el token, en la tabla de **AspNetUsers**, el usuario debe tener el campo **CanRequestToken** en **true**.

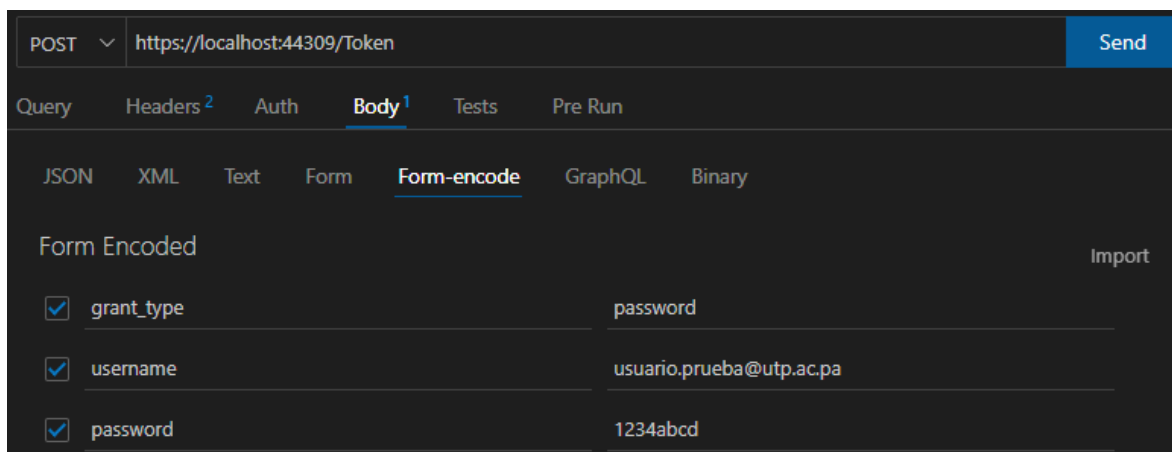
Encabezado.



The screenshot shows the Thunder Client interface with a POST request to `https://localhost:44309/Token`. The 'Headers' tab is selected, showing the following configuration:

Header	Value
User-Agent	Thunder Client (https://www.thunderclient.com)
Accept	application/json

Cuerpo.



The screenshot shows the Thunder Client interface with the same POST request. The 'Body' tab is selected, and the 'Form-encode' format is chosen. The form data is as follows:

Field	Value
grant_type	password
username	usuario.prueba@utp.ac.pa
password	1234abcd

Respuesta.

```
Status: 200 OK   Size: 439 Bytes   Time: 32 ms

Response  Headers10  Cookies  Results  Docs  {}  ≡

1  {
2    "access_token": "zvq6xU7zKsyEhHTb9XZPZ1BSf9fI24hp9DveAupjeejGEcr55UbQpAST4Z9zermZza0Xta1Y9a4z1k
    ms8GgyspGEG4r40WTZE4q3poZ6Y3lKPBgEXPZSGfMvjKHu61FcsfayYsOK1HMT1ovo5-7My
    -xhrWHLUeU4Py0sQn7dk_QvNyB0ie_x0822379qCLj3va506GluDrWpHgZIUG8bltKaYYeGGsGQeKvEbk
    -zI5o2e2upRdJZLB-MF-fqMjGh",
3    "token_type": "bearer",
4    "expires_in": 86399,
5    "userName": "usuario.prueba@utp.ac.pa",
6    ".issued": "Wed, 15 Nov 2023 20:43:52 GMT",
7    ".expires": "Thu, 16 Nov 2023 20:43:52 GMT"
8  }
```

El token expira dentro de un tiempo. Se puede modificar el tiempo de expiración con el que se emiten los tokens, modificando la clave **ExpirationApiToken** que se encuentra en el archivo **Web.config**. El valor debe ser numérico y representa la cantidad de tiempo (en días) que va a durar el token.

Capítulo 4. Autorización.

Después de determina si una petición proviene de un usuario autenticado, en el proceso de **autenticación**, el flujo continúa con el proceso de **autorización**, para determinar si un usuario autenticado, tiene acceso o no a un recurso.

Esta plantilla incluyo un **filtro MVC** personalizado llamado **[AppAuthorization]** para abordar tanto el tema de la **autenticación**, como la **autorización** en un solo lugar. El filtro de se aplica de forma global para todos los controladores en el archivo App_Start/FilterConfig.cs

```
filters.Add(new AppAuthorization());
```

Para su funcionamiento, este filtro hace uso de las siguientes tablas:

- **AspNetUsers**
- **AppOptions**
- **AppUserOptions**

Este filtro personalizado, agregar más funcionalidades que no tiene el filtro [Authorize], que es el predeterminado que incluye el framework MVC 5. **No se recomienda usar el filtro [Authorize]**. El filtro personalizado **[AppAuthorization]** incluye las siguientes funciones:

- Desactivar/Activar el acceso a la plataforma para un usuario determinado. (Ver campo IsActive en la tabla AspNetUser)
- Autorización basada en el modelo **ABAC⁹** (Attribute-Based Access Control)

⁹ [Attribute-Based Access Control](#)

- Control granular para el acceso a las páginas del aplicativo. El acceso se controla con el nombre del controlador, nombre de la acción del controlador, y método HTTP utilizado.
- Posibilidad de **ignorar la autenticación** a nivel de todo un controlador o a nivel de acción específica del controlador. Para ello debe decorar la clase del controlador o el método específico con el filtro **[AllowAnonymous]**
- Posibilidad de **ignorar la autorización** a nivel de todo un controlador o a nivel de acción específica. Para ello debe decorar la clase del controlador o el método específico con el filtro **[NoValidateOptionAccess]**
- Controlar si una opción en específico puede ser **accesible o no mediante API**.
- Manejo personalizado de la respuesta HTTP para peticiones no autenticadas o no autorizadas. Se puede obtener la respuesta tradicional (una página con HTML indicando error HTTP 403) o **una respuesta en formato JSON** indicando también el error HTTP 403. Para recibir la respuesta en JSON se debe incluir el siguiente encabezado en la solicitud:



Creación de opciones de acceso

Para cumplir con el proceso de autorización, primero se deben crear las opciones de acceso, insertando primero en la tabla *AppOptions*. Se describen los campos a continuación:

- **Id:** Clave primaria, el identificador de la opción. No es de auto incremento.
- **Name:** Nombre de la opción
- **Description:** Descripción (opcional) para la opción
- **ParentCategoryId:** El Id de la categoría a la que pertenecerá la opción.
- **OrderInParent:** El orden en que aparecerá (en el menú) la opción dentro de su categoría padre.
- **ControllerName:** Nombre del controlador a la que pertenece la opción.
- **ActionName:** Nombre del método de acción dentro del controlador que representa a la opción
- **HttpMethodId:** Identificador del método HTTP de la acción. Ver tabla AppHttpMethods.
- **ShowInMenu:** Booleano que determina si la opción debe aparecer dentro del menú del aplicativo
- **ApiAccess:** Booleano que determina si se puede acceder a la opción a través de autenticación por token (API)

Creación de categorías

Las diferentes opciones de la aplicación se agrupan en categorías, de igual forma las categorías también pueden agruparse dentro de otra categoría.

Importante: La plantilla define una categoría inicial llamada *“Root”*. **No modifique ni elimine esta categoría** porque es requerida para el correcto funcionamiento de la aplicación.

Para crear una opción, se debe insertar en la tabla AppCategories. Se describen los campos a continuación:

- **Id:** Clave primaria, el identificador de la categoría. No es de auto incremento.

- **Name:** Nombre de la categoría.
- **OrderInParent:** El orden en que aparecerá (en el menú) la categoría dentro de su categoría padre.
- **IconHtmlClass:** Corresponde al valor del atributo “*class*” de HTML que aparecerá sobre el elemento `<i></i>` que se usa para mostrar el ícono. De forma predeterminada esta instalada la librería “Font Awesome Free”, por cuál se pueden utilizar estas clases.
- **ShowInMenu:** Booleano que determina si la categoría debe aparecer dentro del menu del aplicativo.

Capítulo 5. Manejo de la base de datos.

Migraciones

Las migraciones nos permiten gestionar la creación y modificación de la base de datos. Por ejemplo, las tablas, primero se abstraen como clases modelo en el proyecto, y a partir de estos modelos se genera automáticamente el SQL correspondiente para crear las tablas. Igualmente, para realizar cambios a las tablas, primero se hacen los cambios en los modelos y luego se proyectan en las tablas reales.

Creación de tablas.

Para crear una tabla, debemos crear una clase, dentro de la capeta Models/Tables. Dentro de la clase, tenemos que definir las columnas, usando atributos. Hay algunas tablas ya creadas que sirven como ejemplo.

Una vez terminada de crear la tabla, debemos agregarlas a nuestro contexto que se encuentra en el archivo *Models/ApplicationDbContext.cs*

```

public class ApplicationDbContext : IdentityDbContext<ApplicationUser, CustomRole,
{
    1 referencia
    public ApplicationDbContext()
        : base("DefaultConnection")
    {
    }

    15 referencias
    public DbSet<AppCategories> AppCategories { get; set; }
    6 referencias
    public DbSet<AppOptions> AppOptions { get; set; }
    3 referencias
    public DbSet<AppUsersOptions> AppUsersOptions { get; set; }
    0 referencias
    public DbSet<AppTemplates> AppTemplates { get; set; }
    0 referencias
    public DbSet<AppOptionsTemplates> AppOptionsTemplates { get; set; }
    3 referencias
    public DbSet<AppHttpMethods> AppHttpMethods { get; set; }
    0 referencias
    public DbSet<LogUserActivities> LogUserActivities { get; set; }
    1 referencia
    public DbSet<AppReports> AppReports { get; set; }

```

Luego, debemos crear la migración. En la consola de administrador de paquetes, ejecutamos el siguiente comando:

```
Add-Migration UnNombreParaLaMigracion
```

Finalmente, aplicamos la migración a la base de datos con el siguiente comando:

```
Update-Database
```

Modificación de una tablas.

Para modificar una tabla ya existente, simplemente tenemos que hacer los cambios que queramos directamente en el modelo que creamos previamente. Podemos agregar campos, quitar campos, cambiar nombres de los campos, etc.

Luego tenemos que crear una migración para dichos cambios mediante comando en la consola.

```
Add-Migration UnNombreParaOtraMigracion
```

Y finalmente para aplicar los cambios en la base de datos usamos el comando:

```
Update-Database
```

Capítulo 6. Manejo de respuestas HTTP, errores y eventos.

Gestión de respuestas HTTP

Las respuestas HTTP del servidor, principalmente devuelven HTML o JSON, pero también pueden devolver otros tipos.

Dentro de la carpeta **Models**, podemos encontrar el archivo **ResponseModels**, que contiene clases de modelo que se usan para establecer el formato de las respuestas del servidor. Podemos destacar la clase **GeneralResponseModel**, que contiene lo siguiente:

```
public class GeneralResponseModel
{
    8 referencias
    public string Title { get; set; }
    8 referencias
    public string Message { get; set; }
    8 referencias
    public string TypeName { get; set; }
    8 referencias
    public int ErrorCode { get; set; }
    1 referencia
    public IEnumerable<string> ModelErrors { get; set; }
    1 referencia
    public string RedirectUrl { get; set; }
    1 referencia
    public string Html { get; set; }
}
```

- **Title:** Título de la respuesta.
- **Message:** Un mensaje explicativo corto. En caso de errores, se coloca el mensaje de la excepción.
- **TypeName:** El tipo de respuesta. Los 3 valores posibles son: **success**, **warning**, **error**, **info**
- **ErrorCode:** Código del error o la excepción. Usar cero si no hay error.
- **ModelErrors:** Un campo opcional, usado para devolver errores del modelo en la validación de formularios
- **RedirectUrl:** Este atributo solo se debe definir cuando se quiere redireccionar al usuario a alguna URL después de mostrarle el mensaje de respuesta del servidor.
- **Html:** Contenido en formato HTML para mostrar en vez del texto simple de la propiedad "Message". Es opcional.

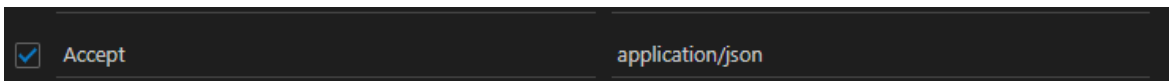
Hay 2 escenarios principales donde se utiliza esta clase:

- La vista **Error.cshtml**, utiliza esta clase como modelo para devolver una página de error personalizada cuando se detecta cualquier tipo de error en la aplicación.
- Para devolver respuestas genéricas para las solicitudes a través de **API**, **XHR** o **fetch**, se debe usar este modelo para construir el JSON de respuesta.

A continuación, se muestra un ejemplo de cómo devolver una respuesta en formato JSON para una solicitud de registrar datos con un formulario.

```
return Json(new GeneralResponseModel { Title = "Completado",
    ErrorCode = 0, Message = "Se agregó la categoría con éxito",
    TypeName = "success" });
```

Nota: Para determinar si una respuesta se tiene que devolver en formato JSON, se verifica el encabezado **Accept**.



Si el valor es **application/json**, la respuesta devolverá un JSON con la estructura de **GeneralResponseModel**, en caso contrario devolverá la respuesta en otro tipo, generalmente HTML.

Gestión de errores

Para la gestión de errores se utiliza el evento global **Application_Error** ubicado en el archivo **Global.asax**. Este evento se dispara cuando ocurre un error en cualquier parte del aplicativo, incluyendo también errores HTTP. Existen algunas excepciones, pero casi la totalidad de errores se pueden capturar mediante este evento.

La lógica necesaria para recibir el error capturado, procesarlo y devolver la respuesta en el formato adecuado, **ya está incluida en esta plantilla**. A menos que sea necesario, **no se recomienda utilizar bloques try catch** en las diferentes clases del aplicativo, es preferible utilizar el mecanismo ya integrado de manejo de errores.

En vez de capturar los errores nosotros mismos, es preferible anticiparnos a los escenarios donde pueda ocurrir un error y **lanzar nosotros mismos la excepción**, para que esta sea manejada por el evento global de errores. Aquellos errores que no podemos anticipar, de igual forma subirán por la pila y serán también manejados por el evento global.

Veamos un ejemplo. En un método que registrar una categoría, no se puede insertar un duplicado en la columna Id, porque generará una excepción de SQL/EF. Puedo validar primero y lanzar una excepción manualmente:

```
public void AddAppCategory(AppCategoriesViewModel appCategory)
{
    if (_adc.AppCategories.Any(a => a.Id == appCategory.CategoryId))
    {
        throw new Exception("Ya existe una categoría con ese ID");
    }
}
```

Luego, esta excepción será manejada automáticamente por los mecanismos antes explicados, y sin que yo tenga que escribir código adicional, se enviará la respuesta del servidor al cliente, con el formato de la clase **GeneralResponseModel** (ya sea en formato de JSON o vista HTML)

Gestión de eventos

Esta plantilla incluye un mecanismo para monitorear eventos de usuario, es decir, llevar un registro de las acciones que realiza un usuario de la aplicación. Para este proceso se hace uso de la librería **log4net**. En esta plantilla, la configuración está habilitada para registrar los eventos en la base de

datos, en la tabla **LogUserActivities**. Es posible registrar eventos en otros medios, como archivos de texto.

Algunos de los eventos que se registran son los siguientes:

- Cuando un usuario inicia sesión.
- Cuando un usuario cierra sesión.
- Cuando un usuario solicita un TOKEN para hacer uso de la API.
- Cuando un usuario autenticado visita una página o realiza una acción.
- Cuando un usuario autenticado ejecuta alguna acción y se produce un error detectado en el servidor (se registra el error)

Los eventos se categorizan en 3 niveles dependiendo del tipo y la gravedad:

- INFO
- ERROR
- WARN

El registro de eventos se puede deshabilitar/habilitar total o parcialmente. Dentro del archivo **Web.config** encontramos las siguientes opciones:

- **EnableLog**: Habilita o deshabilita totalmente el registro de eventos.
- **EnableInfoLog**: Habilita o deshabilita el registro para eventos de tipo **INFO**.
- **EnableErrorLog**: Habilita o deshabilita el registro para eventos de tipo **ERROR**.
- **EnableWarnLog**: Habilita o deshabilita el registro para eventos de tipo **WARN**.

Capítulo 7. Manejo de mensajes y notificaciones.

Esta plantilla ya incluye funciones para mostrar mensajes y notificaciones en las pantallas de la aplicación. Los tipos de mensaje que se puedan mostrar son: **success**, **warning**, **error**, **info**.

Mensajes desde el backend

Dentro de una vista podemos utilizar el helper **FixedNotificationHelper** para llamar al método **RenderAlertMessage**.

```
@using PlantillaDSI.Helpers
@using PlantillaDSI.Models
@{
    ViewBag.Title = "Panel de administración";
    Layout = "~/Views/Shared/Layouts/_Dashboard.cshtml";
}
@FixedNotificationHelper.RenderAlertMessage((SimpleResponseModel)ViewData["SimpleResponse"])
<br />
```

Este método generará un **div** con el mensaje que queremos mostrar. Al método, le tenemos que pasar un objeto de tipo **SimpleResponseModel** (ver archivo **ResponseModels** en la carpeta **Models**) que contiene el tipo y el texto del mensaje. Para este caso, se usa el mecanismo

ViewData, con el que se pasa el objeto de respuesta, desde el controlador a la vista para luego pasarlo al método **RenderAlertMessage**.

Mensajes desde el frontend

Usando JavaScript se puede mostrar un mensaje o notificación desde el frontend. Para ello se usa un componente llamado **src/js/components/ui-message.js**. Este componente se incluye en todas las Layouts (vistas maestras) por lo cual no es necesario volverlo a importar. Se carga en una variable global, llamada **AppGlobalMessenger**, por lo tanto, para mostrar los mensajes, usamos esta variable para llamar al método **showMessage** que muestra el mensaje. Por ejemplo:

```
let responseObj = { TypeName: "error", Message: "Error al  
procesar respuesta" };  
AppGlobalMessenger.showMessage(responseObj, "alert");
```

Podemos ver que el método **showMessage** espera 2 parámetros:

- **responseObject**: El objeto de respuesta que contiene el tipo, mensaje, etc. Se usa el mismo formato de la clase **GeneralResponseModel**.
- **notifyType**: El tipo de notificación. Existen 3 tipos:
 1. **alert**: Mensaje de tipo modal que ocupa el centro de la pantalla para mostrar el mensaje. El usuario debe dar clic en OK o confirmar.
 2. **fixed-notification**: un recuadro en una posición fija de la pantalla que muestra un mensaje. Usa el componente alert de Bootstrap.
 3. **sticky-notification**: Una notificación rápida que se muestra en una esquina de la pantalla y solo duran unos segundos. Transcurrido ese tiempo, se oculta automáticamente.
- **alertDivId** (opcional): El **id** del **div** donde se mostrará el mensaje. Solo aplica cuando se usa *fixed-notification*.

Dialogo de confirmación.

En ocasiones requerimos mostrar un dialogo de confirmación, para preguntar al usuario si está seguro de realizar alguna operación sensitiva. Para estos casos, podemos trabajar con el objeto **AppGlobalMessenger**, haciendo una llamada al método **showConfirmDialog**.

Este método devuelve un objeto *Promise* de JavaScript que se resuelve cuando el usuario confirme o cancele la operación en el cuadro de dialogo. Accediendo a la propiedad *isConfirmed* podemos verificar si el usuario confirmo la operación para continuar con el flujo del programa. Ejemplo:

```
AppGlobalMessenger.showConfirmDialog("Cuidado", "¿Está seguro que  
desea borrar esta categoría?").then((result) => {  
    result.isConfirmed === true &&  
    document.getElementById("formDeleteAppCategory").requestSubmit();  
});
```

Capítulo 8. Convenciones del código.

Las convenciones de código son las reglas para seguir al momento de escribir variables, nombras clases, métodos, etc. En esta plantilla se aplican varias de las convenciones predeterminadas de C# y otras adicionales.

Por ejemplo, para nombrar variables, clases, atributos y métodos, hacemos lo siguiente:

- Los nombres de las clases, atributos públicos de una clase y métodos de una clase deben empezar por una letra y seguir la convención **PascalCase**.
- Los nombres de los atributos privados de una clase deben empezar por el carácter subrayado `_` seguido de la convención **camelCase**.
- Los nombres de las variables deben empezar por una letra y deben seguir la convención **camelCase**.

Para nombrar archivos, módulos, clases, se debe tomar en cuenta lo siguiente:

- Los nombres de las interfaces deben empezar con “I” (mayúscula) seguido de **PascalCase**.
- Los nombres de repositorios deben seguir **PascalCase**, seguido de la palabra **Repository**. Por ejemplo, para un repositorio de productos, sería **ProductsRepository**. Esta misma lógica se aplica para otros tipos de archivos (clases) como Extensions, Helpers y Models.
- Los modelos deben seguir **PascalCase** seguido de la palabra “**Model**” para el nombre del archivo y la clase. Es posible colocar varias clases modelo en un mismo archivo. En ese caso, añadiremos “**Models**” (plural) al nombre del archivo, pero las clases dentro mantendrá la terminación “**Model**”.
- Para dar nombre a los archivos de reportes se debe seguir **PascalCase**.
- Los nombres de las **vistas layout**, y las **vistas parciales** deben empezar con el carácter subrayado seguido de **PascalCase**.
- Los nombres de los archivos y módulos de JavaScript deben estar escritos en minúsculas y seguir la convención **kebab-case**.
- Los nombres de los archivos **scss** deben seguir **kebab-case**. Si el archivo en cuestión es un componente que se va a importar dentro de otro archivo **scss**, al archivo componente debemos colocarle el carácter subrayado al inicio.
- Los parámetros en las URL deben seguir la convención **camelCase**.
- En el código de JavaScript se recomienda usar las características recientes del lenguaje, como el uso de **let** y **const** en lugar de **var**, las “*literal templates*” en lugar de concatenación de cadenas y variables, funciones flecha, etc.

Capítulo 9. Manejo del Frontend

Gestión de paquetes

El manejo del frontend involucra todo lo relacionado a JavaScript, HTML, CSS, SASS, entre otros. Para gestionar paquetes de terceros y sus dependencias utilizaremos NPM.

Los paquetes requeridos por NPM, así como el archivo **package.json** con la configuración necesaria, ya están incluidos en esta plantilla, sin embargo, para poder utilizar los diversos comandos de NPM, es necesario instalar Node.js.

Empaquetado (Module Bundler)

Nuestro proyecto está compuesto diversos archivos de JavaScript, que pueden ser módulos propios, componentes o librerías de terceros. También contamos con archivos de SASS (.scss) que de igual forma se organizan mediante módulos. Este mecanismo nos permite importar dentro de un archivo de JavaScript, los módulos o librerías que necesitamos. Adicionalmente, es posible que tengamos que manejar recursos estáticos, como fuentes, imágenes, gráficos, etc.

Toda esta estructura modular, con importación de paquetes y organización en carpetas, es útil para el programador, ya que le permite tener su código ordenado. Sin embargo, para el navegador web, toda esta organización es irrelevante, el solo actúa como una máquina que ejecuta el JavaScript y CSS proporcionado.

Aquí es donde entra en juego **webpack**, una herramienta que nos permite “compilar”, transformar, optimizar, comprimir y convertir todo nuestro código de JavaScript en un solo paquete que será leído por el navegador. Igualmente, hará lo propio con nuestros archivos de estilo SASS, para generar también un solo paquete optimizado.

Webpack, ya está instalado en esta plantilla, a través de NPM, y su configuración la encontramos en el archivo `webpack.config.js`

Muchas vistas requieren de código de JavaScript para funcionar. En esta plantilla, **a cada una de esas vistas, se le creará una carpeta correspondiente** que contendrá los archivos de JavaScript que se requieran para el funcionamiento de la pantalla asociada a esa vista. Esta carpeta se crea en la ruta **`src/js`**. Por ejemplo:

Si tengo la vista `Views/Authorization/AppCategories.cshtml`

Se debe crear la **carpeta `src/js/appcategories`**

Dentro de la anterior carpeta, **se debe crear un archivo `index.js`** dentro del cual se podrán importar otros módulos o librerías de JavaScript que sean requeridas.

Configuración de webpack para empaquetar módulos

En nuestro archivo `index.js` creado en el paso anterior, es posible que hayamos importado otros archivos, módulos y librerías de terceros. Necesitamos que todo este código (incluyendo las importaciones) se “compile” **en un único archivo**, optimizado que **podamos incluir en el HTML de nuestra vista**.

Para lograr esto, debemos agregar nuestro archivo a la configuración de webpack, agregando el nombre y la ruta, de la siguiente manera:

```

"use strict";

const path = require("path");
const MiniCssExtractPlugin = require("mini-css-extract-plugin");
const CssMinimizerPlugin = require("css-minimizer-webpack-plugin");
const CopyPlugin = require("copy-webpack-plugin");
module.exports = {
  entry: {
    theme: "./src/js/theme/index.js",
    validation: "./src/js/validation/index.js",
    authorization: "./src/js/authorization/index.js",
    appcategories: "./src/js/appcategories/index.js",
    appoptions: "./src/js/appoptions/index.js",
    viewer: "./src/js/viewer/index.js"
  },
  output: {
    filename: "[name].bundle.min.js",
    path: path.resolve(__dirname, "./dist/js")
  },

```

Posteriormente podemos utilizar en la consola, los comando para construir la aplicación.

- Para desarrollo: `npm run build-dev`
- Para desarrollo, y que webpack “vigile” los cambios que hacemos al código y compile automáticamente, usar `npm run watch`
- Para producción: `npm run build-prod`

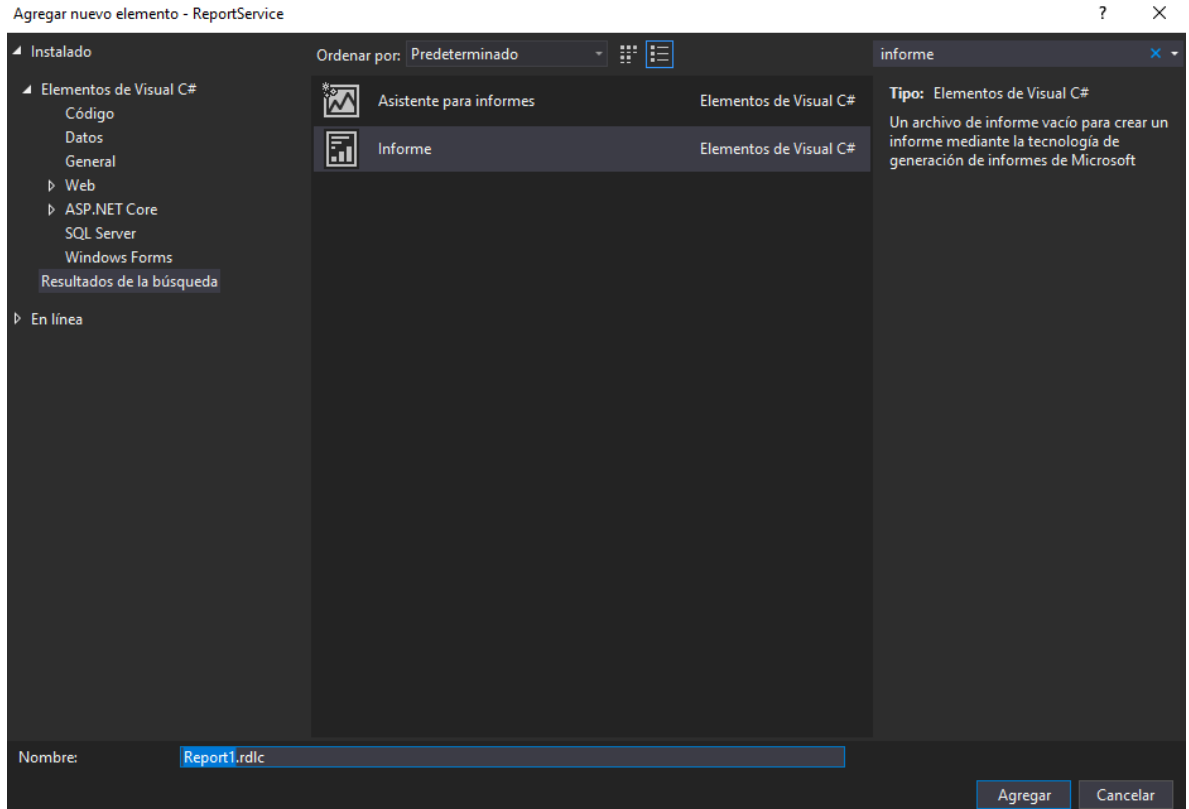
Podremos ver el resultado, en la carpeta **dist/js** el archivo **appcategories.bundle.min.js** con todo el código. Este archivo lo podemos insertar en nuestra vista de la forma habitual mediante una etiqueta `<script></script>` de HTML.

Capítulo 10. Manejo de Reportes

Para el manejo de reportes esta plantilla utiliza **Microsoft Reporting Services**. Para el diseño de los reportes se debe instalar la extensión **Microsoft RDLC Report Designer** para Visual Studio 2019. Los reportes se crean en un proyecto aparte llamado “ReportService” que está dentro de la misma solución.

Un reporte típico tiene los siguientes elementos:

- El archivo del reporte (.rdlc)
En el proyecto de reportes, los archivos rdlc se organizan en carpetas, dentro de la carpeta “Rpt”. Sobre la carpeta damos clic derecho, agregar nuevo elemento. Luego buscamos “informe” en el cuadro de búsqueda. Le colocamos un nombre y damos clic en agregar.



- La clase para el conjunto de datos del reporte (DTO)
Esta clase se usa para abstraer los diferentes campos que tendrá el conjunto de datos que se utilizará en el reporte. Se crea dentro de la carpeta DTO.

Opciones y reportes

Para tener un control sobre el acceso a los reportes, los mismos deben tratar como opciones, es decir, se debe crear en la tabla **AppOptions**. También se debe agregar a la tabla **AppReports**, que extiende la tabla **AppOptions** con columnas específicas para usarse en los reportes, como el nombre del reporte, título, ruta, etc.

Al igual que las demás opciones, los reportes tendrán su respectivo controlador, acción y método (HTTP GET) para acceder al archivo del reporte. Existe un **ReportController** donde se deben crear los métodos de acceso a los reportes. También es requerido pasar el parámetro **reportId** en la URL.

Por ejemplo:

<http://dominiodelsitio.com/Report/GetAppUserOptionRpt?reportId=22>

Importante: El **Id** del reporte (tabla **AppReports**) debe ser el mismo de la opción (tabla **AppOptions**) que se creó para el reporte.

Visor de reportes

Esta plantilla incluye el visor de reportes de Mozilla, basado en la librería PDF.JS que funciona del lado del navegador para visualizar archivos PDF.

Utilizar el visor es fácil, solamente se tiene que llamar a la siguiente dirección URL:

<http://dominiodelsitio.com/Report/Viewer?reportId=22>

Esta dirección cargará la página del visor con el reporte que corresponda según el parámetro **reportId**. Si necesita pasar parámetros adicionales al reporte, se pueden colocar después del parámetro **reportId**.

Visor Modal

También es posible mostrar el visor de reportes en una ventana modal. Primero debemos incluir en la parte superior de nuestra vista, el componente **_ReportViewer.cshtml** (el cual es una vista parcial).

```
@Html.Partial("~/Views/Shared/Components/Report/_ModalReportViewer.cshtml")
```

Dentro del código de JavaScript, importamos el componente pdf-viewer-handler.js i lo inicializamos:

```
import PdfViewerHandler from
"../components/pdf-viewer-handler.js";
const pvh = new PdfViewerHandler(true);
```

Finalmente, llamamos al método **LoadReportOnModal()**, al cual le pasamos la URL del reporte que queremos mostrar.

```
pvh.LoadReportOnModal(reportUrl)
```

Capítulo 11. Manejo de formularios

Existen diversas formas de abordar el tema de los formularios. Se puede realizar de la forma tradicional, escribiendo el HTML necesario, o utilizar las herramientas que nos proporciona el marco MVC para generar el formulario y conectarlo con los modelos y controladores.

A continuación, se sugiere un mecanismo para trabajar con formularios que se envían al servidor de forma asincrónica (usando jQuery AJAX) que hace uso de las herramientas del framework MVC junto con código adicional de JavaScript para abstraer tareas comunes.

Para crear el formulario, se asume que se cuenta con los siguientes elementos ya creados:

- Una vista en blanco.
- Un controlador, con el método que recibe los datos que se envían desde el formulario.
- Nuestro módulo index.js (vacío) para la vista. En el capítulo 9 se explica cómo crear este tipo de módulos.
- El modelo (ViewModel) que representará los datos que envía el formulario.

La vista para el formulario

Dentro de la vista, colocamos el siguiente código:

```
@using PlantillaDSI.Extensions
@model PlantillaDSI.Models.AppCategoriesViewModel
@{
    ViewBag.Title = "Categorías de la aplicación";
    Layout = "~/Views/Shared/Layouts/_Dashboard.cshtml";
}
```

Lo que hacemos aquí es definir el modelo que utilizará el formulario, mediante la palabra **@model**.

Luego procedemos a definir el título de la página, y el **Layout** (_Dashboard.cshtml), es decir la vista maestra de donde se hereda el diseño base para nuestra página y además incluye algunos scripts necesarios.

Luego procedemos a importar como un script, nuestro modulo index.js ya “compilado” (ver capítulo 9)

```
<script src="@Url.ContentVersioned("~/dist/js/appcategories.bundle.min.js")"></script>
```

Posteriormente, usamos el helper **Ajax.BeginForm** que nos permite generar un formulario HTML que funcione con AJAX. Dentro de la segunda llave de apertura, colocaremos el contenido de nuestro formulario (inputs, labels, etc.)

```
@using (Ajax.BeginForm("SaveAppCategory", "Authorization", null, new AjaxOptions
{
    HttpMethod = "POST",
    LoadingElementId = "spinnerAppCategory"
}, new { @id = "formSaveAppCategory" }))
{
    // Form content
}
```

El módulo index.js

En el módulo index.js, importamos los componentes necesarios.

```
import "jquery-validation";
import "jquery-validation-unobtrusive";
import "../components/ajax-form-handler.js";
import FormExtensions from "../components/form-extension.js"
```

El tercer componente (ajax-form-handler.js) es una versión personalizada de la librería **Microsoft.jQuery.Unobtrusive.Ajax**. Esta librería de Microsoft es utilizada para manejar las peticiones AJAX al servidor, de los formularios creados con **Ajax.BeginForm**.

Esta librería ya no recibe soporte por parte de Microsoft. Si bien es cierto que funciona correctamente, presenta algunos inconvenientes desde el punto de vista de la seguridad y el manejo de eventos. Estos inconvenientes se abordan en [este artículo](#)¹⁰, el cual sirvió de base para realizar la versión personalizada de la librería que se usa en esta plantilla.

¹⁰ [What's wrong with default Ajax support in ASP.NET MVC and how to fix it.](#)

Para manejar las diferentes etapas de la petición AJAX al servidor, se agrega un enfoque basado en eventos, que es mas seguro y eficiente que el enfoque original de la librería (que se basa en funciones dentro de los atributos HTML). Los eventos disponibles son los siguientes:

- **“ajax:begin”**: Se dispara justo antes de que se envíe la petición al servidor. Podemos acceder al objeto **event** para leer o modificar algún parámetro de la petición antes de ser enviada. Por ejemplo, si accedemos a **event.detail.settings.url**, podemos modificar la URL y añadir parámetros.
- **“ajax:success”**: Se dispara cuando se recibe la respuesta del servidor a nuestra petición, y la misma fue satisfactoria.
- **“ajax:failure”**: Se dispara cuando la respuesta del servidor a nuestra petición consiste en un error.
- **“ajax-complete”**: Se dispara cuando se completa la petición al servidor, independientemente si fue satisfactoria o terminó en error.

Capítulo 12. Manejo de grid de datos

Para trabajar con grids se recomienda utilizar la librería **datatables** que viene instalada con la plantilla.

En el archivo **src/js/appcategories/index.js** hay un ejemplo de cómo se puede llenar un grid con datos en una vista. Estos datos son cargas mediante una petición AJAX desde el servidor. La librería se encarga de manejar la petición al servidor y solo es necesaria definir los parámetros.

Selección de filas

Una característica común dentro de un grid es la posibilidad de seleccionar una fila y cargar los valores de sus columnas en un formulario. Para manejar la selección de filas, se maneja un enfoque de eventos, en este caso nos interesa el evento **“click”** dentro de las filas de la tabla.

Haciendo uso del objeto **event**, podemos obtener el nodo HTML correspondiente a la fila seleccionada:

```
let trElement = event.target.parentElement;
```

Luego, utilizando la API de la librería podemos obtener un objeto que contenga todos los valores para esa fila, llamando al método **data ()**.

Podemos asignar manualmente cada valor del objeto anterior a su correspondiente “input” del formulario, sin embargo, se recomienda hacer uso del componente **src/js/components/form-extension.js** que debemos importar al principio de nuestro archivo **index.js**.

Con este componente, podemos llamar al método **setFormInputs ()**, pasándole el id del formulario que deseamos llenar con los valores, y el objeto con los valores para asignarlos en una sola línea.

```
fe.setFormInputs("formSaveAppCategory",  
appCategoriesTable.row(trElement).data());
```

IMPORTANTE: Para que lo anterior funcione, cada **“key”** del objeto de valores debe tener el mismo nombre que el atributo **“name”** del input correspondiente en el formulario.

Referencias